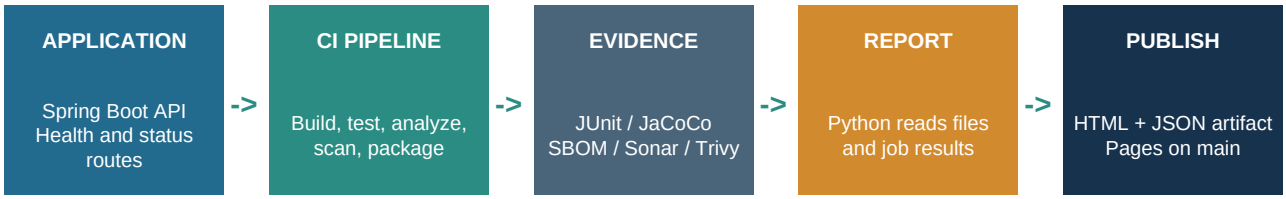
ASPICE CI Architecture

High-level design and job guide for the Spring Boot hackathon solution

Purpose and scope

This document explains how the Spring Boot service, GitHub Actions pipeline, evidence artifacts, Python report generator, and GitHub Pages deployment fit together. The pipeline provides reviewable engineering evidence for a demonstration; it does not establish ASPICE compliance or certification.

Architecture overview



Job isolation	Each GitHub Actions job runs in its own hosted workspace. Producer jobs upload artifacts; the report job downloads them. There is no shared runner filesystem.

Application and container

AspiceDemoApplication starts Spring Boot. HealthController maps GET /api/health and the compatibility route GET /api/status to a JSON response containing application, status, and timestamp. Actuator exposes health and info endpoints. Maven targets Java 21 and packages the service; tests verify the custom health response.

The Dockerfile uses two stages: Maven and a JDK build the application, then a smaller Java runtime image runs the JAR on port 8080. The current CI Docker job downloads a Maven JAR, but the Dockerfile builds another JAR inside its own build stage; it does not copy the downloaded JAR.

CI jobs and execution rules

Jobs without a dependency can run in parallel. A needs dependency normally requires upstream success; job conditions can also skip work.

Job	When / dependency	Purpose and outcome
Requirement Traceability	PR only; Dev environment	Extracts SCRUM- from PR title/body. Jira API checks project SCRUM and the aspice-requirement label. Validates traceability metadata, not code correctness.
Software Bill of Materials	Every run; independent	Trivy filesystem mode creates CycloneDX JSON. Uploads the SBOM artifact for 90 days.
Maven Build	Every run; independent	Runs clean package with tests skipped, verifies a JAR exists, uploads it for 7 days.
JUnit and JaCoCo	Every run; independent	Runs mvn clean verify. Publishes JUnit results and uploads XML test reports plus JaCoCo reports for 90 days.
SonarCloud Analysis	Needs successful tests; Dev	Submits source and coverage analysis. Quality Gate is enforced with sonar.qualitygate.wait=true. Saves measures, gate, and issue JSON when available.
Trivy Filesystem Scan	Every run; independent	Scans configured vulnerability, misconfiguration, and secret categories. Saves JSON. Findings alone do not fail the scan (exit-code 0).
Docker Image Build	Needs Maven Build success	Downloads the JAR artifact, builds the Docker image and saves an image archive for later scan/push jobs.
Trivy Docker Image Scan	Needs Docker Build success	Loads the image and scans vulnerabilities. Saves JSON. Findings alone do not fail the scan (exit-code 0).
Docker Image Push	Non-PR; build and image scan succeed	Uses GITHUB_TOKEN with package write permission to push commit-SHA and branch tags to GHCR. Skipped on PRs.
ASPICE Evidence Report	Always after listed jobs	Downloads available artifacts, runs Python, writes an Actions summary, and uploads the HTML, JSON, and evidence for 90 days.
Deploy HTML Report	Successful report; main push/manual main	Publishes the report to GitHub Pages, downloads it for a content check, and stores a validation copy.

Dependency path

Unit Test -> SonarCloud. Maven Build -> Docker Build -> Trivy Image -> Docker Push (non-PR). SBOM and Trivy Filesystem are independent. The report waits for its declared jobs but uses always() to record partial results. Deployment follows the report.

Evidence flow and report generation

The report job downloads artifacts into evidence/ before the generator runs. The download action creates the destination folders; the workflow does not need a separate mkdir evidence command.

Producer	Output / type	Report location	Report use
JUnit / JaCoCo	XML: TEST-*.xml and jacoco.xml	evidence/tests/	Test cases and coverage metrics
SBOM	JSON: aspice-sbom.cdx.json	evidence/sbom/	CycloneDX components, versions, licenses, package URLs
SonarCloud	JSON: measures, quality gate, issues	evidence/sonar/	Measures, gate conditions, unresolved issues
Trivy filesystem	JSON: trivy-fs-results.json	evidence/trivy/{artifact}/	Filesystem vulnerability, misconfiguration, secret findings
Trivy image	JSON: trivy-image-results.json	evidence/trivy/{artifact}/	Container image vulnerability findings
Docker build / push	JAR, image .tar, job outputs	Artifact handoff / outputs	Image archive feeds scan/push; image tag and digest feed report

Python report generator

scripts/generate_aspice_report.py combines downloaded files under evidence/ with job-result values passed by the workflow. It parses JUnit and JaCoCo XML, plus the SBOM, SonarCloud, and Trivy JSON. Jira issue keys, Docker metadata, and job states arrive through workflow outputs/environment values rather than files in evidence/.

Generated file	Purpose
aspice-report/aspice-report.html	Readable report with pipeline metadata, job results, traceability, tests, coverage, Sonar, SBOM, Trivy, Docker details, evidence map.
aspice-report/aspice-report.json	Structured representation of the same report data for later processing.

	Artifact downloads are allowed to continue when files are missing. The generator generally marks data unavailable or empty so the report can describe a partial run.

Triggers, gates, and operating notes

The event type controls which conditional jobs are enabled. A skipped job often means its condition was not met, rather than an execution failure.

Event	Scope	Expected behavior
Push	main, develop, feature/**, codex/**	Build, tests, scans, and report run. Image push is eligible after build and image scan succeed. Pages deploys only for main.
Pull request	Target main; opened, synchronize, reopened, edited	Requirement check plus build, tests, scans, and report. Docker push and Pages deployment are skipped.
Manual dispatch	Branch selected by the person starting the run	Pipeline runs on the selected branch. Docker push can run after its gates; Pages deployment requires main.

Success and skip behavior

Check	Behavior
Jira traceability	Fails when secrets are unavailable, no SCRUM key is present, the issue cannot be retrieved, the project is not SCRUM, or the aspice-requirement label is missing.
Unit tests / Sonar	Test failure blocks Sonar because Sonar needs the test job. A failed Sonar Quality Gate fails the Sonar job.
Trivy scans	Findings are retained as evidence but do not alone fail scans because exit-code is set to 0. Scanner/action execution errors may still fail.
Docker push	Skipped for pull requests. Requires successful Docker build and image scan, plus packages: write permission.
Report / deployment	Report attempts to run after upstream results. Pages deploys only if report succeeds on a main push or manual main run.

Configuration and boundaries

Jira credentials JIRA_BASE_URL, JIRA_EMAIL, and JIRA_API_TOKEN are expected in the GitHub Dev environment. The workflow hardcodes the Jira project key as SCRUM, so a project-key change requires updating the workflow and PR references. The report job receives Jira issue keys, but does not select Dev; if JIRA_BASE_URL exists only as a Dev environment secret, the report may show keys without clickable Jira links.

The report is engineering evidence for this demonstration. It does not verify that implementation behavior meets a requirement, and it is not an ASPICE compliance or certification determination.